

VÕ TIẾN

Thảo luận kiến thức CNTT trường BK về KHMT(CScience), KTMT(CEngineering)
<https://www.facebook.com/groups/khmt.ktmt.cse.bku>



Kỹ Thuật Lập Trình (Cơ bản và nâng cao C++)

KTILT2 - HK242

TASK 8 Kiến trúc sư OOP - Kiểm soát và tổ chức

Thảo luận kiến thức CNTT trường BK
về KHMT(CScience), KTMT(CEngineering)
<https://www.facebook.com/groups/khmt.ktmt.cse.bku>

Mục lục

1	Đóng gói (Encapsulation)	2
2	Hàm bạn (Friend Function)	4
3	Lớp bạn (Friend Class)	7
4	Kế thừa (Inheritance)	8
4.1	Khái niệm	8
4.2	Các loại kế thừa	9
4.3	Mức độ truy cập trong kế thừa	11
4.4	Hàm tạo và hàm hủy trong kế thừa	13
5	Kế thừa nhiều lớp (Multiple Inheritance)	16
6	Quá tải hàm (Function Overloading)	18
7	Quá tải toán tử (Operator Overloading)	22



1 Đóng gói (Encapsulation)

Khái niệm Đóng gói là một nguyên tắc cơ bản của OOP, nhằm **ẩn chi tiết triển khai bên trong** của một lớp và chỉ cung cấp giao diện (interface) để tương tác với lớp đó từ bên ngoài. Điều này được thực hiện bằng cách:

- Giới hạn quyền truy cập trực tiếp vào các thuộc tính (data members) của lớp.
- Sử dụng các phương thức (methods) để kiểm soát cách dữ liệu được truy cập hoặc thay đổi.
- **Đặc điểm chính:**
 - Kết hợp dữ liệu (attributes) và hành vi (methods) thành một đơn vị (lớp).
 - Sử dụng các mức độ truy cập (public, private, protected) để kiểm soát quyền truy cập.
 - Bảo vệ dữ liệu khỏi bị thay đổi không mong muốn (data hiding).

```
1 class Car {
2     private:
3         string brand; // Thuộc tính bị ẩn
4         int speed;     // Thuộc tính bị ẩn
5     public:
6         // Giao diện công khai để truy cập dữ liệu
7         void setBrand(string b) {
8             brand = b;
9         }
10        void setSpeed(int s) {
11            if (s >= 0) speed = s; // Kiểm soát dữ liệu
12        }
13        string getBrand() {
14            return brand;
15        }
16        int getSpeed() {
17            return speed;
18        }
19    };
```

Mục đích của đóng gói

1. **Ẩn dữ liệu (Data Hiding):** Ngăn chặn việc truy cập trực tiếp vào dữ liệu nội bộ từ bên ngoài lớp, giảm nguy cơ dữ liệu bị hỏng hoặc sử dụng sai.
2. **Kiểm soát truy cập:** Cho phép lớp tự quản lý cách dữ liệu được đọc hoặc ghi, thêm các điều kiện hoặc kiểm tra hợp lệ (validation).
3. **Tăng tính linh hoạt:** Có thể thay đổi triển khai bên trong lớp (ví dụ: cách lưu trữ dữ liệu) mà không ảnh hưởng đến mã sử dụng lớp.
4. **Bảo trì dễ dàng:** Giao diện công khai ổn định giúp mã bên ngoài không cần thay đổi khi nội bộ lớp thay đổi.

Cách triển khai đóng gói

- **Sử dụng mức độ truy cập:**
 - Đặt các thuộc tính ở mức private hoặc protected.
 - Cung cấp các phương thức public (getter và setter) để truy cập hoặc thay đổi dữ liệu.
- **Getter và Setter:**



- **Getter:** Lấy giá trị của thuộc tính.
- **Setter:** Thiết lập giá trị, thường kèm theo kiểm tra hợp lệ.
- **const trong getter:** Nên khai báo getter là const để đảm bảo không thay đổi dữ liệu:

```
1 class Student {
2 private:
3     string name; // Thuộc tính ẩn
4     int age;     // Thuộc tính ẩn
5 public:
6     // Setter với kiểm tra hợp lệ
7     void setName(string n) {
8         if (!n.empty()) name = n; // Chỉ cho phép tên không rỗng
9     }
10    void setAge(int a) {
11        if (a >= 0 && a <= 150) age = a; // Giới hạn tuổi hợp lý
12    }
13    // Getter
14    string getName() {
15        return name;
16    }
17    int getAge() {
18        return age;
19    }
20 };
21
22 int main() {
23     Student s;
24     s.setName("Alice");
25     s.setAge(20);
26     cout << "Name: " << s.getName() << ", Age: " << s.getAge() << endl;
27     s.setAge(-5); // Không thay đổi vì không hợp lệ
28     cout << "Age after invalid set: " << s.getAge() << endl; // Vẫn là 20
29     return 0;
30 }
```



2 Hàm bạn (Friend Function)

Khái niệm Hàm bạn (friend function) là một hàm không phải thành viên (non-member) của lớp, nhưng được khai báo là friend trong lớp để có quyền truy cập vào các thành viên private và protected của lớp đó. Đây là một cơ chế đặc biệt trong C++ nhằm mở rộng quyền truy cập mà không phá vỡ hoàn toàn nguyên tắc đóng gói (encapsulation).

Đặc điểm:

- Không thuộc lớp (không phải phương thức thành viên), nhưng có thể truy cập dữ liệu nội bộ của lớp.
- Được khai báo bằng từ khóa friend bên trong lớp.
- Có thể là hàm độc lập hoặc thuộc một lớp khác.

```
1 class MyClass {
2 private:
3     int secret; // Dữ liệu ẩn
4 public:
5     MyClass(int value) : secret(value) {}
6     // Khai báo hàm bạn
7     friend void printSecret(MyClass obj);
8 };
9
10 // Định nghĩa hàm bạn
11 void printSecret(MyClass obj) {
12     cout << "Secret value: " << obj.secret << endl; // Truy cập private
13 }
14
15 int main() {
16     MyClass obj(42);
17     printSecret(obj); // Output: Secret value: 42
18     return 0;
19 }
```

Mục đích của hàm bạn

1. **Truy cập dữ liệu nội bộ:** Khi một hàm bên ngoài cần làm việc trực tiếp với các thành viên private hoặc protected mà không cần qua getter/setter.
2. **Hỗ trợ tương tác giữa các lớp:** Cho phép một hàm thao tác trên nhiều lớp mà không cần thuộc về bất kỳ lớp nào.
3. **Tối ưu hóa:** Tránh gọi nhiều getter/setter, giúp mã nhanh hơn trong một số trường hợp cụ thể.

Cách sử dụng hàm bạn

1. **Hàm bạn độc lập** Hàm không thuộc lớp nào, được khai báo friend để truy cập thành viên của lớp.

```
1 class Box {
2 private:
3     double width;
4     double height;
5 public:
6     Box(double w, double h) : width(w), height(h) {}
7     // Khai báo hàm bạn
8     friend double getArea(Box b);
```



```
9 };
10
11 // Hàm bạn tính diện tích
12 double getArea(Box b) {
13     return b.width * b.height; // Truy cập private
14 }
15
16 int main() {
17     Box box(5.0, 3.0);
18     cout << "Area: " << getArea(box) << endl; // Output: Area: 15
19     return 0;
20 }
```

2. **Hàm bạn từ một lớp khác** Một phương thức của lớp khác có thể được khai báo là friend để truy cập dữ liệu của lớp hiện tại.

```
1 class Printer; // Khai báo trước vì Printer dùng trong Box
2
3 class Box {
4 private:
5     double width;
6 public:
7     Box(double w) : width(w) {}
8     // Khai báo phương thức của lớp Printer làm bạn
9     friend void Printer::printWidth(Box b);
10 };
11
12 class Printer {
13 public:
14     void printWidth(Box b) {
15         cout << "Width: " << b.width << endl; // Truy cập private
16     }
17 };
18
19 int main() {
20     Box box(10.0);
21     Printer p;
22     p.printWidth(box); // Output: Width: 10
23     return 0;
24 }
```

3. **Hàm bạn với nhiều lớp** Một hàm bạn có thể truy cập dữ liệu của nhiều lớp nếu được khai báo là friend trong tất cả các lớp đó.

```
1 class ClassB; // Khai báo trước
2
3 class ClassA {
4 private:
5     int valueA;
6 public:
7     ClassA(int v) : valueA(v) {}
8     friend int sum(ClassA a, ClassB b);
9 };
```



```
10
11 class ClassB {
12 private:
13     int valueB;
14 public:
15     ClassB(int v) : valueB(v) {}
16     friend int sum(ClassA a, ClassB b);
17 };
18
19 // Hàm bạn cộng giá trị từ hai lớp
20 int sum(ClassA a, ClassB b) {
21     return a.valueA + b.valueB; // Truy cập private của cả hai
22 }
23
24 int main() {
25     ClassA a(5);
26     ClassB b(10);
27     cout << "Sum: " << sum(a, b) << endl; // Output: Sum: 15
28     return 0;
29 }
```

Ưu điểm

- **Linh hoạt:** Cho phép các hàm bên ngoài truy cập dữ liệu mà không cần phá vỡ cấu trúc lớp.
- **Hữu ích trong toán tử:** Thường dùng để định nghĩa các toán tử như +, « cần truy cập dữ liệu nội bộ (ví dụ: friend ostream& operator<<(ostream& os, const MyClass& obj)).
- **Hiệu suất:** Tránh gọi nhiều phương thức công khai khi cần thao tác trực tiếp trên dữ liệu.

Nhược điểm

- **Giảm đóng gói:** Việc cho phép hàm bạn truy cập dữ liệu nội bộ làm suy yếu nguyên tắc ẩn dữ liệu (data hiding).
- **Phụ thuộc:** Hàm bạn trở nên phụ thuộc vào triển khai nội bộ của lớp, nếu lớp thay đổi (ví dụ: đổi tên thuộc tính), hàm bạn cũng phải thay đổi.
- **Dễ lạm dụng:** Sử dụng quá nhiều hàm bạn có thể làm mã khó bảo trì và mất tính mô-đun.

Lưu ý quan trọng

- **Không phải thành viên:** Hàm bạn không thuộc lớp, nên không dùng this và không kế thừa trong các lớp dẫn xuất.
- **Tham số truyền giá trị:** Trong ví dụ trên, đối tượng được truyền bằng giá trị (copy), nếu cần tránh sao chép, dùng tham chiếu (&):

```
1 friend void printSecret(const MyClass& obj);
```



3 Lớp bạn (Friend Class)

Khái niệm

Lớp bạn (friend class) là một lớp được khai báo là friend trong một lớp khác, cho phép tất cả các phương thức của lớp bạn truy cập vào các thành viên private và protected của lớp khai báo nó. Đây là cơ chế giúp hai lớp tương tác chặt chẽ với nhau mà không phá vỡ hoàn toàn nguyên tắc đóng gói (encapsulation).

Đặc điểm:

- Lớp bạn không phải là thành viên của lớp gốc, nhưng có quyền truy cập toàn bộ dữ liệu nội bộ (bao gồm private và protected).
- Được khai báo bằng từ khóa friend theo sau là tên lớp.
- Quan hệ bạn bè là **một chiều** (không đối xứng): nếu lớp A là bạn của lớp B, thì B không tự động là bạn của A trừ khi được khai báo ngược lại.

```
1 class Box {
2     private:
3         double width;
4         double height;
5     public:
6         Box(double w, double h) : width(w), height(h) {}
7         // Khai báo lớp bạn
8         friend class BoxPrinter;
9 };
10
11 class BoxPrinter {
12     public:
13         void printArea(Box b) {
14             double area = b.width * b.height; // Truy cập private
15             cout << "Area: " << area << endl;
16         }
17 };
18
19 int main() {
20     Box box(5.0, 3.0);
21     BoxPrinter printer;
22     printer.printArea(box); // Output: Area: 15
23     return 0;
24 }
```




4 Kế thừa (Inheritance)

4.1 Khái niệm

Kế thừa là một nguyên tắc cốt lõi của lập trình hướng đối tượng (OOP), cho phép một lớp mới (lớp dẫn xuất - derived class) thừa hưởng các đặc điểm (thuộc tính và phương thức) từ một lớp hiện có (lớp cơ sở - base class). Điều này nhằm mục đích tái sử dụng mã, thiết lập mối quan hệ "là một" (is-a), và hỗ trợ tính đa hình.

Mối quan hệ "là một" (is-a)

Ví dụ: "Car là một Vehicle", "Dog là một Animal".

Cú pháp:

```
class DerivedClass : accessSpecifier BaseClass {  
    // Thành viên lớp con  
};
```

Thành viên được kế thừa:

- public và protected: Lớp dẫn xuất có thể truy cập.
- private: Không được kế thừa trực tiếp, nhưng có thể truy cập gián tiếp qua phương thức của lớp cơ sở.

```
1 class Base {  
2 public:  
3     int publicVar; // Có thể truy cập ở mọi nơi  
4  
5 protected:  
6     int protectedVar; // Chỉ truy cập được trong lớp cơ sở và lớp dẫn xuất  
7  
8 private:  
9     int privateVar; // Chỉ có thể truy cập trong lớp cơ sở  
10  
11 public:  
12     // Hàm để gán giá trị privateVar (gián tiếp truy cập private từ lớp dẫn xuất)  
13     void setPrivateVar(int value) {  
14         privateVar = value;  
15     }  
16  
17     // Hàm để lấy giá trị privateVar (gián tiếp truy cập private từ lớp dẫn xuất)  
18     int getPrivateVar() {  
19         return privateVar;  
20     }  
21 };  
22  
23 // Lớp dẫn xuất sử dụng kế thừa public  
24 class DerivedPublic : public Base {  
25 public:  
26     void show() {  
27         cout << "publicVar = " << publicVar << endl; // Truy cập được  
28         cout << "protectedVar = " << protectedVar << endl; // Truy cập được  
29         // cout << "privateVar = " << privateVar << endl; // Lỗi: Không truy cập  
30         // được  
31     }  
32 };
```



Mục đích

Kế thừa là một tính năng quan trọng trong Lập trình Hướng Đối Tượng (OOP) giúp một lớp (lớp con hoặc lớp dẫn xuất) có thể tái sử dụng các thuộc tính và phương thức từ một lớp khác (lớp cha hoặc lớp cơ sở). Mục đích chính của kế thừa:

- **Tái sử dụng mã nguồn:** Tránh việc lặp lại mã bằng cách sử dụng lại các phương thức và thuộc tính từ lớp cha.
- **Mở rộng tính năng:** Lớp con có thể thêm các thuộc tính và phương thức mới mà không cần thay đổi lớp cha.
- **Tổ chức mã theo phân cấp:** Dễ dàng quản lý các lớp có liên quan theo cấu trúc cha - con.
- **Tạo quan hệ "is-a":** Giúp mô hình hóa mối quan hệ giữa các đối tượng (ví dụ: "Xe hơi" là một loại "Phương tiện").

4.2 Các loại kế thừa

Kế thừa đơn (Single Inheritance) Một lớp dẫn xuất kế thừa từ một lớp cơ sở duy nhất.

```
1 class Vehicle {
2 public:
3     int speed;
4     void move() {
5         cout << "Moving at " << speed << " km/h" << endl;
6     }
7 };
8
9 class Car : public Vehicle {
10 public:
11     void honk() {
12         cout << "Honking..." << endl;
13     }
14 };
15
16 int main() {
17     Car car;
18     car.speed = 50;
19     car.move(); // Output: Moving at 50 km/h
20     car.honk(); // Output: Honking...
21     return 0;
22 }
```

Kế thừa đa cấp (Multilevel Inheritance) Một chuỗi kế thừa: lớp A kế thừa từ lớp B, lớp B kế thừa từ lớp C.

```
1 class Animal {
2 public:
3     void eat() {
4         cout << "Eating..." << endl;
5     }
6 };
7
8 class Mammal : public Animal {
9 public:
10     void walk() {
```



```
11     cout << "Walking..." << endl;
12 }
13 };
14
15 class Dog : public Mammal {
16 public:
17     void bark() {
18         cout << "Barking..." << endl;
19     }
20 };
21
22 int main() {
23     Dog dog;
24     dog.eat();    // Từ Animal
25     dog.walk();  // Từ Mammal
26     dog.bark();  // Từ Dog
27     return 0;
28 }
```

Kế thừa đa thừa (Multiple Inheritance) Một lớp dẫn xuất kế thừa từ nhiều lớp cơ sở.

```
1 class Engine {
2 public:
3     void start() {
4         cout << "Engine started" << endl;
5     }
6 };
7
8 class Wheels {
9 public:
10    void roll() {
11        cout << "Wheels rolling" << endl;
12    }
13 };
14
15 class Car : public Engine, public Wheels {
16 public:
17     void drive() {
18         cout << "Driving..." << endl;
19     }
20 };
21
22 int main() {
23     Car car;
24     car.start(); // Từ Engine
25     car.roll();  // Từ Wheels
26     car.drive(); // Từ Car
27     return 0;
28 }
```

Kế thừa phân cấp (Hierarchical Inheritance) Nhiều lớp dẫn xuất kế thừa từ cùng một lớp cơ sở.



```
1 class Shape {
2 public:
3     void draw() {
4         cout << "Drawing a shape" << endl;
5     }
6 };
7
8 class Circle : public Shape {
9 public:
10     void drawCircle() {
11         cout << "Drawing a circle" << endl;
12     }
13 };
14
15 class Square : public Shape {
16 public:
17     void drawSquare() {
18         cout << "Drawing a square" << endl;
19     }
20 };
21
22 int main() {
23     Circle c;
24     Square s;
25     c.draw();          // Từ Shape
26     c.drawCircle();    // Từ Circle
27     s.draw();          // Từ Shape
28     s.drawSquare();    // Từ Square
29     return 0;
30 }
```

Kế thừa lai (Hybrid Inheritance) Kết hợp nhiều loại kế thừa (thường liên quan đến vấn đề kim cương, sẽ giải thích phần sau).

4.3 Mức độ truy cập trong kế thừa

1. Kế thừa public:

- public trong lớp cơ sở → public trong lớp dẫn xuất.
- protected trong lớp cơ sở → protected trong lớp dẫn xuất.

2. Kế thừa protected:

- public trong lớp cơ sở → protected trong lớp dẫn xuất.
- protected giữ nguyên.

3. Kế thừa private:

- public và protected trong lớp cơ sở → private trong lớp dẫn xuất.

4. private không kế thừa: Lớp dẫn xuất không truy cập trực tiếp được thành viên private, cần dùng getter/setter.

```
1 class Base {
2 public:
3     int publicVar;
```



Thành viên trong lớp cơ sở	Kế thừa public	Kế thừa protected	Kế thừa private
public	public	protected	private
protected	protected	protected	private
private	Không kế thừa trực tiếp		

Bảng 1: Mức độ truy cập trong kế thừa

```
4 protected:
5     int protectedVar;
6 private:
7     int privateVar;
8 public:
9     Base() : publicVar(1), protectedVar(2), privateVar(3) {}
10    void setPrivate(int v) { privateVar = v; }
11    int getPrivate() { return privateVar; }
12 };
13
14 class PublicDerived : public Base {
15 public:
16     void access() {
17         publicVar = 10;
18         protectedVar = 20;
19         setPrivate(30);
20         cout << "PublicDerived: " << publicVar << " " << protectedVar << " " <<
21             << getPrivate() << endl;
22     }
23 };
24
25 class ProtectedDerived : protected Base {
26 public:
27     void access() {
28         publicVar = 11;
29         protectedVar = 21;
30         setPrivate(31);
31         cout << "ProtectedDerived: " << publicVar << " " << protectedVar << " " <<
32             << getPrivate() << endl;
33     }
34 };
35
36 class PrivateDerived : private Base {
37 public:
38     void access() {
39         publicVar = 12;
40         protectedVar = 22;
41         setPrivate(32);
42         cout << "PrivateDerived: " << publicVar << " " << protectedVar << " " <<
43             << getPrivate() << endl;
44     }
45 };
46
47 int main() {
48     PublicDerived pub;
49     ProtectedDerived pro;
50     PrivateDerived pri;
```



```
49     pub.publicVar = 100; // OK: public inheritance
50     pub.access();       // Output: PublicDerived: 10 20 30
51
52     // pro.publicVar = 200; // Lỗi: protected inheritance
53     pro.access();       // Output: ProtectedDerived: 11 21 31
54
55     // pri.publicVar = 300; // Lỗi: private inheritance
56     pri.access();       // Output: PrivateDerived: 12 22 32
57
58     return 0;
59 }
```

4.4 Hàm tạo và hàm hủy trong kế thừa

Trong kế thừa, khi một đối tượng của lớp dẫn xuất (derived class) được tạo hoặc bị hủy, cả lớp cơ sở (base class) và lớp dẫn xuất đều có vai trò trong việc khởi tạo và dọn dẹp tài nguyên. Điều này dẫn đến sự phối hợp giữa hàm tạo và hàm hủy của các lớp trong chuỗi kế thừa.

- **Hàm tạo (Constructor):**

- Dùng để khởi tạo đối tượng, bao gồm cả các thành viên kế thừa từ lớp cơ sở.
- Trong kế thừa, hàm tạo của lớp cơ sở được gọi trước, sau đó đến hàm tạo của lớp dẫn xuất.

- **Hàm hủy (Destructor):**

- Dùng để dọn dẹp tài nguyên (như bộ nhớ động) khi đối tượng bị hủy.
- Trong kế thừa, hàm hủy của lớp dẫn xuất được gọi trước, sau đó đến hàm hủy của lớp cơ sở.

Thứ tự gọi hàm tạo

- **Quy tắc:** Hàm tạo được gọi từ lớp cơ sở thấp nhất trong chuỗi kế thừa lên đến lớp dẫn xuất cuối cùng.
- **Cơ chế:**
 - Khi tạo một đối tượng của lớp dẫn xuất, trình biên dịch tự động gọi hàm tạo của lớp cơ sở trước.
 - Nếu lớp cơ sở có tham số, lớp dẫn xuất phải cung cấp giá trị cho hàm tạo của lớp cơ sở thông qua danh sách khởi tạo (initializer list).

```
1  class Base {
2  public:
3      Base() {
4          cout << "Base default constructor" << endl;
5      }
6      Base(int x) {
7          cout << "Base parameterized constructor with " << x << endl;
8      }
9  };
10
11 class Derived : public Base {
12 public:
13     Derived() {
14         cout << "Derived default constructor" << endl;
15     }
16     Derived(int x) : Base(x) { // Gọi hàm tạo của Base với tham số
```



```
17     cout << "Derived parameterized constructor" << endl;
18     }
19 };
20
21 int main() {
22     cout << "Creating Derived with default constructor:" << endl;
23     Derived d1;
24     cout << "\nCreating Derived with parameterized constructor:" << endl;
25     Derived d2(10);
26     return 0;
27 }
```

Output:

Creating Derived with default constructor:
Base default constructor
Derived default constructor

Creating Derived with parameterized constructor:
Base parameterized constructor with 10
Derived parameterized constructor

Giải thích:

- Với d1, hàm tạo mặc định của Base được gọi trước, sau đó là Derived.
- Với d2, lớp Derived gọi hàm tạo có tham số của Base qua danh sách khởi tạo, sau đó thực thi thân hàm tạo của mình.

Thứ tự gọi hàm hủy

- **Quy tắc:** Hàm hủy được gọi ngược lại với hàm tạo, từ lớp dẫn xuất xuống lớp cơ sở.
- **Cơ chế:** Khi một đối tượng ra khỏi phạm vi (scope) hoặc bị delete, trình biên dịch đảm bảo tài nguyên của lớp dẫn xuất được dọn dẹp trước, sau đó mới đến lớp cơ sở.

```
1  class Base {
2  public:
3      Base() {
4          cout << "Base constructor" << endl;
5      }
6      ~Base() {
7          cout << "Base destructor" << endl;
8      }
9  };
10
11 class Derived : public Base {
12 public:
13     Derived() {
14         cout << "Derived constructor" << endl;
15     }
16     ~Derived() {
17         cout << "Derived destructor" << endl;
18     }
19 };
20
21 int main() {
22     Derived d;
```



```
23     return 0;  
24 }
```

Output:

```
Base constructor  
Derived constructor  
Derived destructor  
Base destructor
```

Giải thích:

- Hàm tạo: Base -> Derived.
- Hàm hủy: Derived -> Base. Điều này đảm bảo tài nguyên của lớp dẫn xuất được giải phóng trước khi lớp cơ sở bị hủy.

Hàm tạo với tham số và danh sách khởi tạo Nếu lớp cơ sở có hàm tạo yêu cầu tham số, lớp dẫn xuất phải cung cấp giá trị thông qua danh sách khởi tạo. Nếu không, trình biên dịch sẽ báo lỗi.

```
1  class Person {  
2  protected:  
3      string name;  
4  public:  
5      Person(string n) : name(n) {  
6          cout << "Person constructor: " << name << endl;  
7      }  
8  };  
9  
10 class Student : public Person {  
11 private:  
12     int id;  
13 public:  
14     Student(string n, int i) : Person(n), id(i) { // Gọi hàm tạo của Person  
15         cout << "Student constructor: ID " << id << endl;  
16     }  
17 };  
18  
19 int main() {  
20     Student s("Alice", 123);  
21     return 0;  
22 }
```




5 Kế thừa nhiều lớp (Multiple Inheritance)

Khái niệm Kế thừa nhiều lớp là khả năng một lớp dẫn xuất (derived class) thừa hưởng các đặc điểm (thuộc tính và phương thức) từ nhiều lớp cơ sở (base classes) cùng lúc trong C++. Điều này cho phép một lớp kết hợp các tính năng từ nhiều nguồn khác nhau, thể hiện mối quan hệ "là một" với nhiều thực thể.

```
1 class Derived : public Base1, public Base2 {
2     // Nội dung lớp dẫn xuất
3 };
```

Một lớp dẫn xuất có thể "là một" của nhiều lớp cơ sở (ví dụ: một AmphibiousVehicle vừa là Car vừa là Boat).

Ưu điểm của kế thừa nhiều lớp

1. **Kết hợp chức năng:** Cho phép một lớp tích hợp các đặc điểm từ nhiều nguồn (ví dụ: vừa có khả năng chạy trên đất liền vừa trên nước).
2. **Tái sử dụng mã linh hoạt:** Tận dụng các lớp cơ sở đã có mà không cần viết lại hoặc tổ hợp phức tạp.
3. **Hỗ trợ mô hình hóa phức tạp:** Phù hợp với các tình huống thực tế nơi một thực thể có nhiều vai trò (ví dụ: một nhân viên vừa là Employee vừa là TaxPayer).

```
1 class Employee {
2     protected:
3         string name;
4     public:
5         Employee(string n) : name(n) {}
6         void work() { cout << name << " is working" << endl; }
7 };
8
9 class TaxPayer {
10     protected:
11         double income;
12     public:
13         TaxPayer(double i) : income(i) {}
14         void payTax() { cout << "Paying tax on " << income << endl; }
15 };
16
17 class Manager : public Employee, public TaxPayer {
18     public:
19         Manager(string n, double i) : Employee(n), TaxPayer(i) {}
20         void manage() {
21             work();
22             payTax();
23             cout << name << " is managing" << endl;
24         }
25 };
26
27 int main() {
28     Manager mgr("Alice", 80000);
29     mgr.manage();
30     return 0;
31 }
32 // Output:
```



```
33 // Alice is working
34 // Paying tax on 80000
35 // Alice is managing
```

Nhược điểm và thách thức

1. **Độ phức tạp tăng:** Việc quản lý nhiều lớp cơ sở làm mã khó đọc và dễ xảy ra lỗi.
2. **Xung đột tên (Name Collision):** Nếu hai lớp cơ sở có thành viên trùng tên, lớp dẫn xuất phải giải quyết xung đột bằng cách chỉ định rõ ràng (dùng ::).
3. **Vấn đề kim cương (Diamond Problem):** Khi hai lớp cơ sở kế thừa từ cùng một lớp gốc, lớp dẫn xuất có thể nhận nhiều bản sao của lớp gốc, gây xung đột hoặc trùng lặp dữ liệu.



6 Quá tải hàm (Function Overloading)

Khái niệm Quá tải hàm là một tính năng trong C++ cho phép định nghĩa nhiều hàm cùng tên trong cùng một phạm vi (scope), miễn là các hàm này khác nhau về danh sách tham số (số lượng, kiểu dữ liệu, hoặc thứ tự tham số). Điều này giúp tăng tính linh hoạt và dễ đọc của mã bằng cách cho phép một tên hàm thực hiện nhiều hành vi khác nhau tùy theo cách gọi.

Đặc điểm:

- Tên hàm giống nhau, nhưng chữ ký (signature) phải khác nhau.
- Chữ ký bao gồm: số lượng tham số, kiểu dữ liệu tham số, và thứ tự tham số (không bao gồm kiểu trả về).
- Trình biên dịch chọn hàm phù hợp dựa trên đối số (argument) được truyền khi gọi.

```
1 class Example {
2 public:
3     void print(int x) {
4         cout << "Integer: " << x << endl;
5     }
6     void print(double x) {
7         cout << "Double: " << x << endl;
8     }
9     void print(string s) {
10        cout << "String: " << s << endl;
11    }
12 };
13
14 int main() {
15     Example ex;
16     ex.print(5);           // Gọi print(int)
17     ex.print(3.14);       // Gọi print(double)
18     ex.print("Hello");    // Gọi print(string)
19     return 0;
20 }
21 // Output:
22 // Integer: 5
23 // Double: 3.14
24 // String: Hello
```

Mục đích của quá tải hàm

- **Tăng tính dễ đọc (Readability):** Dùng cùng tên hàm cho các tác vụ liên quan thay vì đặt tên khác nhau (ví dụ: printInt, printDouble).
- **Tính linh hoạt (Flexibility):** Cho phép xử lý nhiều kiểu dữ liệu hoặc tình huống khác nhau mà không cần logic điều kiện phức tạp.
- **Hỗ trợ đa dạng giao diện (Interface Variety):** Cung cấp nhiều cách gọi hàm phù hợp với nhu cầu người dùng.
- **Tái sử dụng tên (Name Reuse):** Tận dụng tên hàm quen thuộc trong các ngữ cảnh khác nhau.

Cách hoạt động Chữ ký hàm (Function Signature):

- **Trình biên dịch phân biệt các hàm dựa trên danh sách tham số, không dựa trên kiểu trả về.**
- Ví dụ: void print(int) và int print(int) không được coi là quá tải vì chỉ khác kiểu trả về.



```
1 class Calculator {
2 public:
3     int add(int a, int b) {
4         return a + b;
5     }
6     double add(double a, double b) {
7         return a + b;
8     }
9     string add(string a, string b) {
10        return a + b;
11    }
12    int add(int a, int b, int c) { // Quá tải với 3 tham số
13        return a + b + c;
14    }
15 };
16
17 int main() {
18     Calculator calc;
19     cout << "Int: " << calc.add(3, 4) << endl; // Output: Int: 7
20     cout << "Double: " << calc.add(3.5, 2.5) << endl; // Output: Double: 6
21     cout << "String: " << calc.add("Hello ", "World") << endl; // Output: String:
22     // Hello World
23     cout << "Three ints: " << calc.add(1, 2, 3) << endl; // Output: Three ints: 6
24     return 0;
25 }
```

Ẩn hàm (Function Hiding) Nếu lớp dẫn xuất định nghĩa một hàm cùng tên với lớp cơ sở nhưng khác chữ ký, các phiên bản quá tải trong lớp cơ sở sẽ bị ẩn (hide) trừ khi được đưa vào phạm vi bằng using.

```
1 class Base {
2 public:
3     void print(int x) {
4         cout << "Base int: " << x << endl;
5     }
6     void print(double x) {
7         cout << "Base double: " << x << endl;
8     }
9 };
10
11 class Derived : public Base {
12 public:
13     using Base::print; // Đưa các hàm từ Base vào phạm vi
14     void print(string s) {
15         cout << "Derived string: " << s << endl;
16     }
17 };
18
19 int main() {
20     Derived d;
21     d.print(5); // Base int: 5
22     d.print(3.14); // Base double: 3.14
23     d.print("Hello"); // Derived string: Hello
24     return 0;
25 }
```



25 }

Liên hệ với kế thừa nhiều lớp Trong kế thừa nhiều lớp, quá tải có thể phức tạp hơn nếu các lớp cơ sở có hàm cùng tên. Cần dùng :: hoặc using để giải quyết.

```
1 class A {
2 public:
3     void process(int x) { cout << "A int: " << x << endl; }
4 };
5
6 class B {
7 public:
8     void process(double x) { cout << "B double: " << x << endl; }
9 };
10
11 class C : public A, public B {
12 public:
13     void process(string s) { cout << "C string: " << s << endl; }
14 };
15
16 int main() {
17     C c;
18     c.A::process(5);    // A int: 5
19     c.B::process(3.14); // B double: 3.14
20     c.process("Hello"); // C string: Hello
21     return 0;
22 }
```

Lưu ý quan trọng

- **Kiểu trả về không ảnh hưởng:** Quá tải không dựa trên kiểu trả về, chỉ dựa trên danh sách tham số.

```
1 void func(int x) {}
2 // int func(int x) {} // Lỗi: trùng chữ ký, chỉ khác kiểu trả về
```

- **Tham số mặc định (Default Arguments)** Có thể gây nhầm lẫn khi kết hợp với quá tải.

```
1 void func(int x) { cout << "One: " << x << endl; }
2 void func(int x, int y = 0) { cout << "Two: " << x << " " << y << endl; }
3
4 int main() {
5     func(5); // Lỗi: không rõ ràng (gọi func(int) hay func(int, int)?)
6     return 0;
7 }
```

- **Ép kiểu (Type Conversion)** Trình biên dịch có thể chọn hàm gần khớp nhất nếu không có khớp chính xác.



```
1 void print(int x) { cout << "Int: " << x << endl; }
2 void print(double x) { cout << "Double: " << x << endl; }
3
4 int main() {
5     print(5.5f); // Gọi print(double) qua ép kiểu từ float
6     return 0;
7 }
8 // Output: Double: 5.5
```

- **Hiệu suất** Quá tải không gây chậm vì được giải quyết tại thời điểm biên dịch (compile-time).



7 Quá tải toán tử (Operator Overloading)

Khái niệm

Quá tải toán tử là kỹ thuật trong C++ cho phép bạn gán ý nghĩa mới cho các toán tử có sẵn khi chúng được sử dụng với các đối tượng của lớp do bạn định nghĩa. Điều này giúp mã trở nên trực quan và giống với cách sử dụng toán tử trên các kiểu dữ liệu cơ bản (như int, double).

Đặc điểm:

- Toán tử được định nghĩa lại thông qua hàm thành viên hoặc hàm bạn (friend function).
- Cú pháp: `return_type operator(symbol)(parameters)`.
- Không thay đổi ưu tiên (precedence) hoặc tính kết hợp (associativity) của toán tử.

```
1 class Point {
2 private:
3     int x, y;
4 public:
5     Point(int x = 0, int y = 0) : x(x), y(y) {}
6     // Quá tải toán tử +
7     Point operator+(const Point& other) {
8         return Point(x + other.x, y + other.y);
9     }
10    void display() {
11        cout << "(" << x << ", " << y << ")" << endl;
12    }
13 };
14
15 int main() {
16     Point p1(2, 3), p2(4, 5);
17     Point p3 = p1 + p2; // Gọi operator+
18     p3.display();       // Output: (6, 8)
19     return 0;
20 }
```

Mục đích của quá tải toán tử

1. **Tăng tính trực quan:** Cho phép sử dụng toán tử theo cách tự nhiên (ví dụ: `p1 + p2` thay vì `p1.add(p2)`).
2. **Tính nhất quán:** Làm cho lớp của bạn hoạt động giống kiểu dữ liệu cơ bản.
3. **Hỗ trợ biểu thức phức tạp:** Dễ dàng viết các biểu thức như `obj1 + obj2 * obj3`.
4. **Tính linh hoạt:** Mở rộng chức năng của toán tử cho các kiểu dữ liệu mới.

Cách triển khai quá tải toán tử

1. **Là hàm thành viên** Toán tử được định nghĩa trong lớp, với đối số đầu tiên là đối tượng hiện tại (qua `this`).

```
1 class Complex {
2 private:
3     double real, imag;
4 public:
5     Complex(double r = 0, double i = 0) : real(r), imag(i) {}
6     // Quá tải toán tử + làm hàm thành viên
7     Complex operator+(const Complex& other) {
```



```
8         return Complex(real + other.real, imag + other.imag);
9     }
10    void display() {
11        cout << real << " + " << imag << "i" << endl;
12    }
13 };
14
15 int main() {
16     Complex c1(3, 2), c2(1, 4);
17     Complex c3 = c1 + c2;
18     c3.display(); // Output: 4 + 6i
19     return 0;
20 }
```

2. **Là hàm bạn (Friend Function)** Dùng khi toán tử cần truy cập trực tiếp vào thành viên private của hai đối tượng, hoặc khi đối số đầu tiên không phải là đối tượng của lớp.

```
1 class Complex {
2 private:
3     double real, imag;
4 public:
5     Complex(double r = 0, double i = 0) : real(r), imag(i) {}
6     friend Complex operator+(const Complex& a, const Complex& b);
7     void display() {
8         cout << real << " + " << imag << "i" << endl;
9     }
10 };
11
12 Complex operator+(const Complex& a, const Complex& b) {
13     return Complex(a.real + b.real, a.imag + b.imag);
14 }
15
16 int main() {
17     Complex c1(3, 2), c2(1, 4);
18     Complex c3 = c1 + c2;
19     c3.display(); // Output: 4 + 6i
20     return 0;
21 }
```

Các toán tử thường được quá tải

- Toán tử số học: +, -, *, /, %.
- Toán tử so sánh: ==, !=, <, >, <=, >=.
- Toán tử gán: =.
- Toán tử nhập/xuất: «, ».
- Toán tử tăng/giảm: ++, --.
- Toán tử truy cập: [], () .

```
1 class Vector {
2 private:
3     int x, y;
```




```
4 public:
5     Vector(int x = 0, int y = 0) : x(x), y(y) {}
6     Vector operator+(const Vector& v) { return Vector(x + v.x, y + v.y); }
7     bool operator==(const Vector& v) { return x == v.x && y == v.y; }
8     friend ostream& operator<<(ostream& os, const Vector& v);
9 };
10
11 ostream& operator<<(ostream& os, const Vector& v) {
12     os << "(" << v.x << ", " << v.y << ")";
13     return os;
14 }
15
16 int main() {
17     Vector v1(1, 2), v2(3, 4), v3(1, 2);
18     Vector v4 = v1 + v2;
19     cout << v4 << endl;           // Output: (4, 6)
20     cout << (v1 == v3) << endl;  // Output: 1 (true)
21     cout << (v1 == v2) << endl;  // Output: 0 (false)
22     return 0;
23 }
```

Kế thừa toán tử từ lớp cơ sở

```
1 class Base {
2 protected:
3     int value;
4 public:
5     Base(int v = 0) : value(v) {}
6     Base operator+(const Base& other) {
7         return Base(value + other.value);
8     }
9     void display() { cout << value << endl; }
10 };
11
12 class Derived : public Base {
13 public:
14     Derived(int v = 0) : Base(v) {}
15 };
16
17 int main() {
18     Derived d1(5), d2(10);
19     Derived d3 = d1 + d2;  // Sử dụng operator+ từ Base
20     d3.display();          // Output: 15
21     return 0;
22 }
```

Lưu ý quan trọng

1. **Không thể quá tải một số toán tử** :: (phạm vi), . (truy cập thành viên), .*, sizeof, typeid, alignof.
2. **Ít nhất một toán hạng phải là kiểu người dùng định nghĩa**: Không thể quá tải `int + int`, nhưng có thể quá tải `MyClass + int`.



3. **Giữ ngữ nghĩa tự nhiên:** + nên biểu thị cộng hoặc kết hợp, không nên dùng cho trừ để tránh nhầm lẫn.
4. **Trả về tham chiếu khi cần:** Với toán tử như =, «, trả về tham chiếu để hỗ trợ chuỗi phép toán (chaining).

```
1 class Number {
2 private:
3     int value;
4 public:
5     Number(int v = 0) : value(v) {}
6     Number& operator=(const Number& other) {
7         value = other.value;
8         return *this; // Trả về tham chiếu để chaining
9     }
10    friend ostream& operator<<(ostream& os, const Number& n);
11 };
12
13 ostream& operator<<(ostream& os, const Number& n) {
14     os << n.value;
15     return os;
16 }
17
18 int main() {
19     Number n1(5), n2, n3;
20     n3 = n2 = n1; // Chaining nhờ trả về tham chiếu
21     cout << n3 << endl; // Output: 5
22     return 0;
23 }
```